

Debriefing et conseils pour l'examen

Voici quelques conseils pour l'examen, à destination des étudiant·es en difficulté.

Chemins

Les notions de chemins relatifs et absolus doivent être assimilées à 100% depuis la L1. Si votre compréhension de ces choses aussi basiques n'est pas totale, vous ne pourrez pas aborder des questions plus subtiles.

- les questions sont *typées* : `cd ../plop` n'est pas un chemin, c'est une commande. Si on vous demande un chemin, alors un chemin est attendu.
- `../../plop` n'est pas la même chose que `/../../plop`
- `~/var/spool/mail/root` ou `./var/spool/mail/root` ou `/var/spool/mail/root/` (trailing slash) ne désignent pas le même chemin.
- si vous ne connaissez pas le HOME de l'utilisateur courant, alors vous ne connaissez pas la valeur de `~`. Le HOME de l'utilisateur `toto`, qui est accessible via `~toto`, n'est pas forcément `/home/toto`. Pensez à votre HOME sur les machines de salle TP. Regardez aussi qui est `~man`, `~root` ou `~www-data` sur votre conteneur par exemple.
- lorsque vous devez désigner un fichier, utilisez son chemin absolu, ça permet d'éviter les confusions, car des fichiers différents peuvent avoir un même nom.

Vocabulaire

On s'attend à ce que vous utilisiez un vocabulaire précis : lien physique, lien symbolique, contenu du fichier, inode, nom, chemin, répertoire, système de fichiers, block device, ...

Ambiguïté

Lorsque vous argumentez, ne multipliez pas les arguments dans l'espoir que l'un d'entre eux soit le bon.

- par exemple, si vous argumentez en même temps sur le nombre de liens nommant le répertoire courant et sur le nombre de liens nommant un fichier particulier, alors que seul le premier comptage est pertinent, le mauvais argument à propos du nombre de liens nommant le fichier particulier décrédibilisera l'argument correct. Un argumentaire doit être complet et sans fioritures.
- si une opération sur un fichier n'est pas possible parce qu'il manque une permission, soyez précis sur la permission manquante (quel fichier (précisez son chemin), quelle permission (lecture, écriture, exécution), et quel propriétaire (l'utilisateur propriétaire, le groupe propriétaire, others))
- de même, si vous exprimez qu'une commande est possible, précisez exactement les permissions qui vous permettent de le déduire (et pas « l'utilisateur toto a toutes les permissions nécessaires »).
- si une opération sur un fichier n'est pas possible parce qu'il manque une permission en lecture, n'écrivez pas « le fichier ne donne que la permission en écriture », car cela laisse penser que la permission de lecture et la permission d'exécution sont toutes les deux nécessaires à l'opération.
- si vous hésitez entre `./plop` ou `/plop` ou `~/plop`, il est inutile de faire commencer le chemin par une rature artistique en espérant que la personne qui corrige sélectionnera le bon préfixe.
- lorsqu'on vous demande une commande, essayez de mettre une commande complète, par exemple dans la commande `chmod u+x` il manque un nom de fichier.
- Pour les questions à réponse oui/non pour lesquelles on vous demande de justifier, la justification est obligatoire (on ne joue pas à pile ou face).
- lorsqu'on vous demande de justifier un choix de commande, faites-le. La clarté de la justification peut permettre à la personne qui corrige d'apprécier la justesse de votre raisonnement même si la syntaxe de la commande est incorrecte.

Itération dans une sous-arborescence et globbing

La meilleure façon d'itérer dans une sous-arborescence est d'utiliser `find`.

Si on veut changer les permissions de tous les fichiers dont le nom finit en `.sh` dans la sous-arborescence de `/opt`, la commande `chmod -R <permission> /opt/*.sh` ne fonctionnera pas car le mécanisme de globbing du shell va remplacer `/opt/*.sh` par les chemins de tous les fichiers qui se trouvent (directement) dans le répertoire `/opt/` et dont le nom finit par `.sh`, mais pas les fichiers qui se trouvent dans des sous-répertoires de `/opt/`. Le remplacement de `/opt/*.sh` pas la liste des fichiers est faite par le shell avant l'exécution de l'exécutable `chmod`.

Quelques erreurs fréquentes (en vrac)

- « le fichier `fic1` n'a pas la permission d'écrire ». Les fichiers n'ont pas de permissions (ils n'agissent pas, ils ne peuvent pas écrire), éventuellement on peut dire qu'ils « donnent » la permission en écriture à tel user du système.
- si `fic1` est un lien symbolique vers le fichier régulier `fic2`, la commande `cat fic1` affichera le contenu du fichier régulier `fic2`, car pour un lien symbolique, il y a **déréférencement**. Si on veut afficher le contenu du lien symbolique `fic1`, c'est à dire le chemin vers le nom de fichier `fic2`, alors on il faut utiliser la commande `readlink fic1`.
- ne pas confondre lien symbolique et lien physique. En particulier un lien symbolique est un fichier, qui est différent du fichier vers lequel il pointe.
- inventer des règles :
 - si `fic` est un fichier régulier et `rep` est un répertoire, certain-es étudiant-es pensent qu'il n'est pas possible d'exécuter la commande `cp fic rep`, parce que la seule utilisation possible de l'exécutable `cp` serait `cp fic1 fic2` où `fic1` est un fichier qui existe et `fic2` est un nom de fichier qui n'existe pas encore. Il faut comprendre qu'un exécutable peut avoir plusieurs comportements selon le type des arguments donnés dans la commande. Lorsque `rep` est un répertoire qui existe, la commande `cp fic rep` va copier le fichier `fic` dans le répertoire `rep`, le nouveau fichier aura comme chemin `rep/fic`. Ne pas connaître cette utilisation de l'exécutable `cp` révèle un manque de pratique car ce genre d'utilisation est assez fréquent.
 - penser que `cp fichier.txt fichier2` ne peut pas marcher parce que `fichier2` doit avoir la même extension que `fichier1.txt`.
- ça n'est pas parce que 2 fichiers ont les mêmes méta données (type de fichier, taille, permissions, user et groupe propriétaires, timestamps, etc), autre que le numéro d'inode, que ce sont les mêmes fichiers.
- en sortie de commande `ls -l`, la taille indiquée au niveau d'un répertoire est la taille de son contenu (un ensemble de liens), ça n'est pas la somme des tailles des contenus des fichiers nommés par le répertoire. Si on voit souvent des tailles de 4 KiB, c'est parce que le système de fichiers réserve de la place parce que les liens d'un répertoire évoluent souvent, et cela évite de devoir allouer de la place à chaque ajout d'un lien.
- le fait d'être caché est lié au nom du fichier (commencer par un point), cela n'a aucun impact sur les permissions. « le fichier est caché donc j'ai pas le droit », ou « le fichier est caché donc il n'y a que l'user qui peut le voir » ne peuvent pas être des arguments valables.
- SSH : la paire de clefs du client ne sert que pour l'authentification, pas pour le chiffrement de la connexion.

Les pertes de temps

Ces choses là ne sont pas fausses, mais n'emportent pas forcément confiance de la personne qui corrige pour la suite.

- si vous voulez créer un fichier régulier `f` avec le contenu `hop` vous pouvez directement l'éditer avec un éditeur de texte ou faire un `echo hop > f`. Dans les deux cas, il est inutile de créer `f` auparavant (avec la commande `touch`).

- si vous copiez un fichier **a** vers un fichier **b**, il est inutile de créer le fichier **b** au préalable (avec la commande **touch**).
- (plutôt lors des TP) si vous vous trouvez dans votre HOME et que vous voulez éditer le fichier `~/.ssh/config` sans rien faire d'autre dans le répertoire `.ssh`, il est inutile de se déplacer,

```
$ vim .ssh/config
```

est bien plus court à taper (surtout avec l'autocomplétion) que

```
$ cd .ssh
```

```
$ vim config
```

```
$ cd ..
```